Name (LN, FN):	Quizon, Marc Lesley E.	Date:	**Jan 28 2026**
Section:	S18	Grade:	

6.0 Unix / Linux Command Line Interface (CLI) Basics – Processes, Unix Shells and Shell Scripts

The Unix system allows job control because each command or program executed in the Unix system is a considered a process. Job control allows running of processes, killing processes or creating background processes. The "`ps`" command **shows which processes are running** on the system. This like the "`ls`" command for processes. Running processes can also be **terminated** using the "`kill`" command. Job control will be discussed more in the course.

The Unix shell is a program that allows interactive use of the Unix operating system and the computer hardware. There are different shells created for the Unix system. The first shell was the **Bourne shell**. After the Bourne shell, the Korn and C shell was later created. The **Korn shell** is a superset of the Bourne shell while the C shell was "C" like ("C" as in the C programming language). Although there are several shells available, the shells have the same set of features with only subtle differences. For example, the "history" command is not available in the Bourne shell but is available in the Korn and C shells. The prompts of different shells also look different.

Today, the Linux system uses the **Bourne Again shell**. This "new" shell has the useful features of Korn and C shells. You could say that it is the hybrid of the Korn and C shell. Do remember that the Bourne Again shell is only in Linux versions of Unix systems.

As mentioned earlier, these shells have same set features. These features are aliasing, environment setting, variables and shell programming or also known as shell scripts. These features will be discussed in better detail in this section.

Shell scripts allow a user to make useful utilities using Unix commands or other utilities. Shell scripts also have constructs like programming languages that allow flexibility.

With a minimum background of shells, scripts and job control, the objectives of this section are:

- Familiarization with Linux and Unix shells, it features and how to use it.
- Familiarization with Linux / Unix shell scripts and how create and use shell scripts.
- Familiarization with controlling jobs on the Linux / Unix system.

6.1. Processes

One of the abilities of the Unix system is run several programs at a time. This is called **multi-tasking**. The Unix system is the first system to provide this feature. Each process running on the Unix system has **a process ID**.

Since there are several programs running at the same time, there must be a way to control these programs or processes. There are several types of processes on the Unix system. Two basic types of processes is the **parent and child process**. As the name implies, the parent spawns a child process. If the parent process terminates before the child process returns, the **child process will become an orphan**. If the child process does not return, it is **a zombie or defunct process**. Also, the "**daemon**" process has the process ID of 1 which is the "main" parent of the processes in the Unix system.

Users can control the process using the "`ps`" and "`kill`" command; that allow a user to control processes. When the "`ps`" command is executed, something similar to the table below will be seen on the screen.

```
 PID   TTY        TIME   CMD
1361   pts/    00:00:01  bash
       0
1832   pts/    00:00:00  ps
       0
```

The first column shows the **process id**. The second column shows **which terminal is running it** while the third column shows **how much processing time** is being used by the process. Fourth column shows the name or the **command name**.

The "`kill`" command can be used to **terminate** a process. When the "`kill`" command is used, make sure you know the **process ID**. There are two ways to kill a process, soft kill and hard kill. With the soft kill, processes are terminated **properly** while the hard kill terminates a process **abruptly**.

Step 1 On the prompt, type the command "`ps`".

1a. How many processes are running? What are the processes that you see?

> There are 2 processes running, bash and ps

1b. On the prompt type the command "`ps -e`". What are the additional processes for?

> With the -e option, we are telling ps to show every process in the system including those running in the background and from other users. These additional processes are the system and background processes that keep our Linux running.

1c. On the prompt type the command "`ps -f`". Is the number or columns different from 1a? What is the "`f`" option for?

> There are more columns displayed. The -f option stands for full format, the extra columns we see are User ID (UID), Parent Process ID (PPID), CPU Utilization (C), and Start Time (STIME)

1d. On the prompt type the command "`ps -e | more`". What does the command do?

> This command will not show all the additional processes in the system in one go. The user can press the spacebar to continue viewing more processes.

1e. On the prompt type the command "`ps -e | grep ss`". What does the command do?

> This command filters the list of processes shown to those that only contain 'ss'.

Step 2 On the prompt enter the command "`sleep 1000`" and open another remote session to same server. Do not exit your first remote session.

2a. Using the new remote session at the prompt, enter the command "`ps`". Were you able to see the "`sleep`" program?

> No, I only see bash and ps.

2b. Using the new remote session, at the prompt, enter the command "`ps -u <username>`". Were you able to see the "`sleep`" program? What is the process ID of the "`sleep`" program?

> Yes I can see the sleep program, its PID is 913958.

2c. At the prompt, kill the "`sleep`" program using the "`kill <pid>`" command. Was the "sleep" program terminated?

> Yes it was terminated. I was able to verify this by checking the results of "`ps -u <username>`", the sleep program is no longer there. Moreover, a 'terminated' message was shown in the first remote session

2d. Log-out of the new remote session. If ever the process cannot be killed using the soft kill command, use the command "`kill -9 <pid>`" (hard kill). Before using the command, use the soft kill first because it terminates a process properly than the hard kill command.

Step 3 At the prompt, enter the command "`sleep 1000 &`".

3a. What is the output of the command?

```
lbysyads18user20@SYSSVR1:~$ sleep 1000 &
[1] 913994
lbysyads18user20@SYSSVR1:~$
```

The output we get refers to *[job number] process id*

The "`&`" at the end of the command tells the shell to make it a background process.

3b. At the prompt, enter the command "`ps -f`". Looking at the "`PPID`" column, what is the parent ID of the process of the "`sleep`" process? Why was it the parent?

> The parent ID of the sleep process is 913922, this is actually the process ID of the '-bash' command. Since we made 'sleep' a background process, it's parent would be the 'bash' command.

3c. Kill the sleep program.

Step 4 At the prompt, enter the command "`ps -f`". Record the process ID of the current shell. ***(913922)***

4a. At the prompt, open a new shell using the command "`bash`" and then enter the command "`sleep 1000 &`". Use the command "`ps -f`" to check the processes. What is the parent ID of the "`sleep`" program?

```
lbysyads18user20@SYSSVR1:~$ kill 913994
lbysyads18user20@SYSSVR1:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
lbysyad+  913922  913921  0 18:24 pts/17   00:00:00 -bash
lbysyad+  914000  913922  0 18:48 pts/17   00:00:00 ps -f
[1]+  Terminated              sleep 1000
lbysyads18user20@SYSSVR1:~$
```

```
lbysyads18user20@SYSSVR1:~$ bash
lbysyads18user20@SYSSVR1:~$ sleep 1000 &
[1] 914008
lbysyads18user20@SYSSVR1:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
lbysyad+  913922  913921  0 18:24 pts/17   00:00:00 -bash
lbysyad+  914001  913922  0 18:50 pts/17   00:00:00 bash
lbysyad+  914008  914001  0 18:50 pts/17   00:00:00 sleep 1000
lbysyad+  914009  914001  0 18:50 pts/17   00:00:00 ps -f
lbysyads18user20@SYSSVR1:~$
```

The parent ID of the sleep program is 914008. This is actually the process ID of the new shell we prompted.

4b. Exit the new shell by using the command "`exit`". Enter the command "`ps -f`". What is the parent ID of the "`sleep`" process? Which process does the parent ID point to?

> The parent ID of the sleep process is 1. This is because the process that spawned the sleep program was exited before it ended. Hence, its parent is now the daemon process which is the "main" parent of the processes in the Unix system.

Since the process that spawned the sleep program was terminated before the sleep program ended, the sleep program is now called an orphan. Notice that the daemon process adopted the sleep program.

4c. Kill the "`sleep`" process that was orphaned. Were you able to kill it?

> Yes, I was able to kill it

6.2. The Unix Shell

As mentioned earlier, the shell allows an interactive use of the Unix system and the computer hardware. Without it, it will be hard to use the Unix system and computer. There are different shells available in the Unix system. These shells have same set of features with only little differences. Most of the time, their differences are how the commands are entered.

6.2.1. Bourne Shell (bash), Bourne Again Shell (bash) and C Shell (csh)

The Bourne Again (bash) and C (csh) shell are different shell available on the Linux and Unix version. The mentioned shells have different behaviors. The Linux system uses the **Bourne Again Shell by default** but other Linux systems may use the Bourne shell (bsh). The Bourne Again shell incorporates useful features of the **Korn** shell (ksh), which is a widely used shell in Unix shells before, and the **C** shell. The Bourne Again shell was developed by the Free Software Foundation.

Unix systems uses the C shell or the TC shell (tcsh). The C shell is almost the same as the Bourne Again shell. One difference is the **absence of using the cursor keys** for cycling through command history in the C shell. The TC shell has that feature. The C shell is shown in this session for familiarity when the user uses a Unix system.

Step 5 On the prompt, enter the command "`csh`". Observe what happens to the prompt. Always take note that you are in the C shell.

5a. What does your prompt your look like? Why do you think the prompt changed? (In this case, the usual prompt for the C shell in most Unix systems is "`%`" but in Linux, it has a different prompt.)

```
Last login: Wed Jan 28 18:38:16 2026 from 112.208.231.60
lbysyads18user20@SYSSVR1:~$ csh
%
```

This is what my prompt looks like, it changed because we typed csh which brings us to the C shell.

5b. On the prompt, enter the following commands: "`ls -l`", then "`ls`". Try using the up and down cursor keys. Were you able to call the previous commands? Why or why not?

No I was not able to retrieve the previous commands. This is because the C shell does not have this feature

5c. Clear the command prompt and type the command "`history`". What was the output?

```
% history
%
```

There was no output

5d. On the prompt, type the command "`!!`". What does the command do?

```
% history
% !!
history
%
```

This command displayed the previous command I typed.

5e. On the prompt, type the command "`touch this_is_a_file_with_a_long_name.txt`". Try to display the file using the "cat this_" then press the "tab" key. What happened? Did the shell complete the filename? Why or why not?

No the shell did not complete the file name. This is because C shell is old and does not have this feature

Step 6 Type the command "`exit`" at the command prompt and enter the command "`bash`".

6a. Did the prompt change? What does the prompt look like?

```
% exit
% exit
lbysyads18user20@SYSSVR1:~$ bash
lbysyads18user20@SYSSVR1:~$
```

The prompt went back to the usual prompt

6b. On the prompt, enter the following commands: "`ls -l`", then "`ls`". Try using the up and down cursor keys. Were you able to call the previous commands? Why or why not?

Yes we are able to do this. This is because the Bourne Again Shell has this feature

6c. On the prompt, type the command "`history`". What was the output?

The output is a long list of the commands that I typed

6d. On the prompt, type the command "`cat this_`" then press the "`tab`" key. What happened? Did the shell complete the filename? Why do you think so?

Yes the shell completed the filename. This is because the Bourne Again Shell has this feature.

Step 7 Type the command "`exit`" at the command prompt.

7a. On the prompt, enter the following commands: "`ls -l`", then "`ls`". Try using the up and down cursor keys. Were you able to call the previous commands? Why or why not?

Yes I am able to call the previous command. This is because the Bourne shell has this feature.

7b. Clear the command prompt, and then enter the command "`history`". What was the output?

It is similar to the Bourne Again Shell. I see all of the commands that I typed before

7c. On the prompt, type the command "`touch testfile`" and then enter the command "`vi !*`" or "`nano !*`". Which file was opened by the text editor? What is the "`!*`" for?

'testfile' was opened by the text editor. '!' works by taking the arguments from the previous command except the command itself and apply it to the current command we typed. So basically, our 'vi !*' in this case is the same as 'vi testfile'

7d. Exit the text editor. On the prompt, type the command "`cat this_`" then press the "`tab`" key. What happened? Did the shell complete the filename? Why or why not?

The shell completed the filename, it worked because the Bourne Shell has this feature.

6.2.2. Shell Features

"alias" Command

An alias allows a way to **give a different name or shorter name for a command.** This allows a user to "personalize" some commands that are used often.

Step 8 On the prompt, enter the command "`alias ll="ls -l"`". Again on the prompt, enter the command "`ll`".

8a. What is the output of the "`ll`" command? Is it the same as the "`ls -l`" command?

It is the same as the "ls -l" command

8b. On the prompt, type the command "`unalias ll`". Enter the command "`ll`" at the prompt. What is the output? Where you able to show the listing of your directory? Why or why not?

The shell said "command not found". We are unable to see the listing of our directory because we unaliased our command.

8c. What does the "`alias`" and "`unalias`" command do?

"alias" will create an alias. Whereas, "unalias" will remove an alias that was created.

"set" Command

The set command allows **setting and unsetting of flags that control** shell-wide characteristics. There are a lot of flags available on the Unix system. What will be presented are the useful ones.

Step 9 At the prompt, enter the command "`ls > list`" and then enter the command "`cat list`".

9a. At the prompt, enter the command "`ls -l > list`" and then enter the command "`cat list`". Was the text file overwritten? Why or why not?

The file was overwritten. This is because we used '>', this operator will overwrite existing files. If we just want to append the file, we should use ">>"

9b. At the prompt, enter the command "`set -o noclobber`" and then enter the command "`ls > list`". Were you allowed to overwrite the file? What does the "`set -o noclobber`" command do?

No, the shell did not allow me to overwrite the file. Firstly, the "-o" is used to turn on an option. Secondly, noclobber is a mode that prevents existing files to be overwritten. This command simply turns on this feature so that existing files do not get overwritten.

9c. At the prompt, enter the command "`set +o noclobber`" and then enter the command "`ls > list`". Were you allowed to overwrite the file? What does the `set +o noclobber`" command do? Why was the "`+`" symbol used instead of "–"?

Yes we were able to overwrite the file. The command turns off the noclobber option. The difference is that '-o' is to turn on, while '+o' is to turn off. The "+" was used because we wanted to turn off the noclobber mode

Step 10 At the prompt, enter the command "`set -o verbose`" and then enter the command "`ls -l`".

10a. What does the "`set -o verbose`" command do? What happens to the output of the "`ls -l`" command when verbose mode is on?

> The command's output still displayed as usual. However this time, the command itself (ls -l) was printed at the top before the main output

10b. At the prompt, enter the command "`set +o verbose`" and then enter the command "`ls -l`". Did it turn off the verbose mode? How were did you verify it?

> Yes it turned off verbose mode, I was able to verify it because the command is no longer shown at the top before the main output

Command Sequencing

Several commands can be executed in succession in the Unix system. By using the ";", commands can be **sequenced together**. The Unix system is also able to do **conditional sequencing.** With conditional sequencing, a command will only be executed **if the first one is successful**, or the next command will only be executed if there was an error.

Step 11 At the prompt, type the command "`date; ls -l`".

11a. What did the command do? Did it execute two commands in succession?

> Yes it executed both command in succession.

11b. At the command prompt, type the command "`date; ls; pwd > out.txt`". View the text file `using` a text editor or "`cat`" command. Describe the contents of the "`out.txt`" file.

> The out.txt file contains the result of the 'pwd' command.

```
lbysyads18user20@SYSSVR1:~$ cat out.txt
/home/lbysyads18user20
lbysyads18user20@SYSSVR1:~$
```

11c. At the command prompt, type the command "`(date; ls; pwd) > out.txt`". View the text file using a text editor or "`cat`" command. Describe the contents of the "`out.txt`" file.

> The content contains the result of all the commands within the ( )

```
lbysyads18user20@SYSSVR1:~$ (date; ls; pwd) > out.txt
lbysyads18user20@SYSSVR1:~$ cat out.txt
Wed Jan 28 22:13:49 PST 2026
coffees
dir1
dir2
lbysyadsuser20
list
long_list
out.txt
rfc1034.pdf
rfc2186.txt
testfile
this_is_a_file_with_a_long_name.txt
/home/lbysyads18user20
```

11d. Why were the output of 11b and 11c different? What made them different even though the same commands were used?

> The main difference is the usage of the ( ), this groups commands together. In 11c, we grouped the 3 commands together. Hence, output.txt will contain the result from all 3 commands

Step 12 At the prompt, type the command "`mkdir dir3`" then enter the command "`ls -d dir3 && echo "Directory found"`". Notice that this is a conditional command. The "`&&`" specifies that the next command will be executed when the first command is successful.

12a. Did it execute the second command?

> Yes it executed the second command successfully.

12b. At the prompt, type the command "`ls -d dir4 && echo "Directory found"`". Did it execute the second command? Why or why not?

> No, the result is a 'no such file or directory" error message. The second command did not perform because the first command was not successful

Shell Variables

Shell variables are **stored information** required by processes so that processes can function properly. An example of a variable is how your command prompt looks like. The value is stored in a variable is called PS1. There are two types of shell variables: **local and global (also known as environment variable).** Local variables are variables that are available for **the current shell session** only while global variables are variables that are available to the **current and all child or sub-shells** that the user or the system might start

Step 13 At the prompt, enter the command "`my_name="<your first name>"`" and then enter the command "`echo $my_name`".

13a. What was the output of the "`echo $my_name`" command? Were you able to create a variable?

> Yes I was able to create a variable

```
ls: cannot access 'dir4': No such file or directory
lbysyads18user20@SYSSVR1:~$ my_name="Marc"
lbysyads18user20@SYSSVR1:~$ echo $my_name
Marc
lbysyads18user20@SYSSVR1:~$
```

13b. At the prompt, enter the command "`echo my_name`". Was the output same as 13a? Is the "`$`" sign needed to display the value of the variable? Why or why not?

> The output is simply "my_name". The $ sign is needed to indicate a variable. Without it, the text is simply plain text

13c. What did you create? A local or global variable?

> It is a local variable

13d. At the prompt, enter the command "`bash`" and then enter the command "`echo $my_name`". Were you able to see you name?

> No, this is because we created a local variable. So, it is only visible in the current shell session

13e. Without exiting the shell yet, type the command "`my_name="<your first name> <your last name>"`" and enter the command "`echo $my_name`". Were you able to see the variable value? Explain why you were not able to see your name earlier in 13d.

> Yes I can see the variable value. This was not visible earlier because we created a local variable, which is not accessible to another shell session. Hence, the value was not seen earlier

13f. Exit the shell and enter the command "`echo $my_name`". Was the output same as 13e? Why or why not?

No it is not the same. This is because they are separate and completely different variables on different shells. Hence, their values are independent from each other.

13g. At the prompt, enter the command "`export my_name`" and then enter the command "`bash`". Display the variable "`$my_name`" by using the command "`echo $my_name`". Is the output the same with 13f? Why or why not?

Yes the output is the same. This is because of the "export my_name" command. This created a global variable which means that we can access it on different shell sessions.

13h. What did you create? Local or global variable?

Global variable

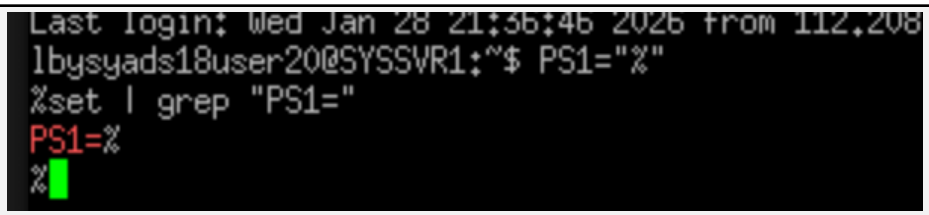
Step 14 Type "`exit`". At the command prompt, enter the command "`PS1="%"`".

14a. What happened to the prompt? Did it change?

Yes it changed. The prompt is now "%"

Notice that PS1 is a shell variable. It was created by the operating system. In this case, the variable PS1 holds the text or command what the prompt should look like.

14b. At the prompt, enter the command "`set | grep "PS1="`". What did it display? Were you able to find the PS1 assignment statement? *Note: The variable assignments are found at the top of the page and are arranged alphabetically by variable name.*



The set command simply displays all shell variables. I was able to find the PS1 assignment statement

14c. At the command prompt, enter the command "`PS1="\\u@\\h:\\W\\$"`". Did the command prompt go back to its original state?

Yes the command prompt went back to the original state.

There are special characters that the shell can accept for the prompt setting. These settings are:

Setting	What for
\u	Username
\d	Date
\h	Hostname
\$	Dollar Sign
\W	Working directory

14d. Using the command "`set | grep <username>`", find the shell variable for your *log-in name*. Is there a shell variable for it? What is the name of the variable?

Yes there is a shell variable for it. The variable is LOGNAME

```
%PS1="\\u@\\h:\\W\\$"
lbysyads18user20@SYSSVR1:~$set | grep lbysyads18user20
HISTFILE=/home/lbysyads18user20/.bash_history
HOME=/home/lbysyads18user20
LOGNAME=lbysyads18user20
PWD=/home/lbysyads18user20
USER=lbysyads18user20
lbysyads18user20@SYSSVR1:~$
```

14e. Using the command "`set | grep <username>`", find the shell variable for your *home directory*. Is there a shell variable for it? What is the name of the variable?

Yes the variable name is HOME

Initialization Files

Initialization files or scripts can contain series of commands, settings and variables. Initialization files allows a user to **personalize some settings.** There are two types of initialization files: **system wide and personal**. The system wide initialization is set by the **administrator** and is the **default** initialization file. The personal initialization file resides in the **user's home directory** and can be "personalized" by the **user**. In the bash shell, there are two initialization files, the **".bash_profile" or ".profile" and the ".bashrc**". The first initialization file can contain shell **variable** settings while the second initialization file contains **alias** settings. The ".bash_profile" can also contain other command that can be executed during the logging in.

Step 15 At the prompt, enter the command "`PS1=%`". Make sure that the prompt has changed and enter the "`exit`" command to log out of the terminal.

15a. Login to the server again and check the prompt. Were you able to change the prompt? Did it go back to its original state?

Yes the prompt changed to %. After exiting and logging back in, the prompt is back to the original state

15b. Edit the "`.profile`" file using a text editor. At the end of the file, enter the text "`PS1="[\u@\h \W]$"`". Save the file and exit. Login to the server again and check the prompt. Did the prompt change permanently?

I am able to write the file. The result is that the prompt would change permanently

Step 16 At the prompt, enter the command "`alias lx="ls -lR"`". Enter the command "`lx`".

16a. Were you able to see the effect of the alias?

Yes I am able to see the effect. By simply typing lx, we get the result for "ls -lR"

16b. Exit the session and login to the server again. Enter the command "`lx`" at the prompt. Were you able to see the effect of the command? Why do you think the command was not successful?

No, this is because the alias will only work for the current shell session

16c. Edit the "`.profile`" using a text editor. At the end of the file, enter the text "`alias lx="ls -lR"`". Save the text file and exit. Login to the server again and check the command. Were you able to use the alias? Why so?

I have permission to edit the file. The result is that the alias will be saved for future shell sessions and will work successfully.

6.3. Shell Scripts

A shell script (sometimes also known as shell program) is **a series of commands and utilities** that have been put to a file by using a text editor. When the shell script is executed, the commands are interpreted and **executed** by the shell program. The commands in the shell script are executed by the shell one after the other. A shell script is like any other programming language and has its own syntax. You can **define variables; assign various values, and use control and iterative statements.**

After creating the shell script, the file permission of the shell script should be **read and execute** (at least by the owner). So that the shell can find your shell script, make a directory called "**bin**" in your home directory. Your shell script should be saved in the "bin" directory. The shell will be able to find it because the shell variable "PATH" enumerates "$HOME\BIN" as one of the directories to look for executables files for commands entered. You will see this later in this section.

The programming language like syntax of the shell scripts is easy to understand if you have experiences with the C programming language. One problem though is that you **should be using the C shell**. With the bash shell, the syntax is a little different but still easy to understand. For this section of the course, the bash shell syntax will be used. The syntax for shell scripting in the bash and C shell is provided as a supplemental material in this course. Please get it from your instructor.

A Shell Script Program

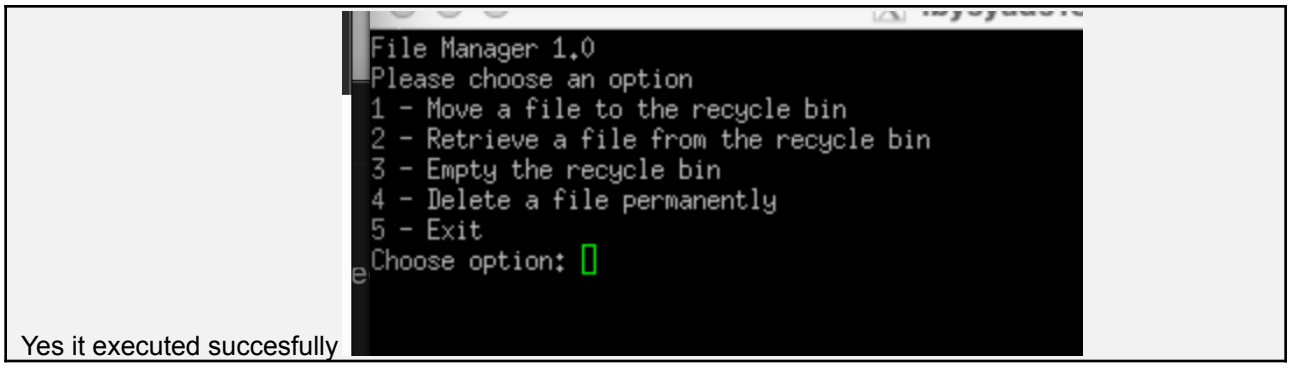
Step 17 Type the following text and save it your directory. Use the filename "`fmgr`".

```
get_file_name()
{
  echo -n "Enter filename: "
  read mfile
}

print_no_file()
{
  echo "$mfile: No such file or directory"
}

while [ 1 ]
do
  clear
  echo "File Manager 1.0"
  echo "Please choose an option"
  echo "1 - Move a file to the recycle bin"
  echo "2 - Retrieve a file from the recycle bin"
  echo "3 - Empty the recycle bin"
  echo "4 - Delete a file permanently"
  echo "5 - Exit"
  echo -n "Choose option: "
  read option
  case $option in
    1)
      get_file_name
      if [ -f $mfile ]; then
```

```
          mv $mfile $HOME/.junk
         else
          print_no_file
         fi;;
       2)
         get_file_name
         if [ -f $HOME/.junk/$mfile ]; then
          mv $HOME/.junk/$mfile $HOME
         else
          echo "$mfile: No such file or directory"
         fi;;
       3)
         echo -n "Are you sure you want to empty recycle bin? (y/n)"
         read answer
         if [ $answer="y" -o $answer="Y" ]; then
          rm $HOME/.junk/*
         fi;;
       4)
         get_file_name
         if [ -f $mfile ]; then
          rm -i $mfile
         else
          print_no_file
         fi;;
       5)
         exit;;
       *)
     esac
done
```

After saving the file, add the "`execute`" permission. Also, create a directory called "`.junk`". The period (.) at the start of the filename will create a hidden file..

17a. Execute the shell script by entering "`./fmgr`" on the prompt. Did it execute?



Yes it executed succesfully

17b. Choose option number 5, this will exit the script. Execute the script again but this time, use the command "`fmgr`". Did it execute? What was the message?

No, the message states "command not found" . It did not execute because ./ is needed for the shell to recognize and execute programs in the current directory

Remember that commands are actually programs in the Unix system. The shell automatically looks for them in certain directories specified in the shell variable "`$PATH`". If you want to see the paths, just enter the command "`echo $PATH`".

Step 18 Create a directory called "`bin`" on your home directory and type in the command "`PATH=$PATH:$HOME/bin`". The previous command adds the "`$HOME/bin`" to one of the paths to look for progams. Move the "`fmgr`" file to the "`bin`" directory in your home directory. Execute the "`fmgr`" script and make sure you are in your home directory (~).

18a. Were you able to execute the file? Why were you able to execute the file even if it wasn't in the home directory?

Yes we were able to execute the file successfully. This is possible because we added a new file path for the shell to look for programs. Hence, it was able to locate and execute fmgr successfully

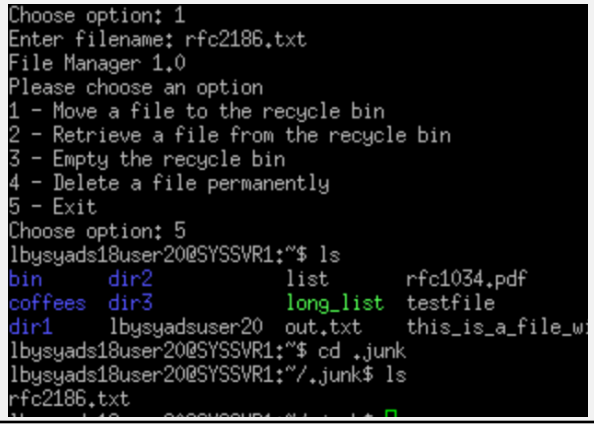
Step 19 Copy the files "`rfc1034.pdf`" and "`rfc2186.txt`" from the "`/home/files`" directory. If you still have the files, there is no need to copy it. Execute the "`fmgr`" script file. **In answering the questions in this step, make sure you point out line/s of code in the script file to justify your answers.**

19a. Put the file "`rfc2186.txt`" to the recycle bin and exit the script file. Is the file in the "`.junk`" directory?

Yes the file is in the .junk directory. This worked because of

```
get_file_name
if [ -f $mfile ]; then
mv $mfile $HOME/.junk
```

The function get_file_name simply gets input from the user. With -f, we are checking if the file exists. Then, if exists the shell will execute the command mv $mfile (variable that holds our input) $HOME/.junk



19b. Execute the "`fmgr`" script file and retrieve the "`rfc2186.txt`" file. Is the file still in the "`.junk`" directory? Were you able to retrieve it?

No, the file is no longer in the .junk directory. It is back in the home directory. I was able to retrieve it. This is because of

```
if [ -f $HOME/.junk/$mfile ]; then
        mv $HOME/.junk/$mfile $HOME
```

It first checks if the file exists in the .junk directory. Then if exists, it will mv the file back to the home directory

19c. Execute the "`fmgr`" script file and empty the recycle bin. Is the file still in the "`.junk`" directory?

No, the file is no longer in the recycle bin. This is because of

```
if [ $answer="y" -o $answer="Y" ]; then
        rm $HOME/.junk/*
```

So, when we type y or Y. It will delete the contents of the .junk directory by using the rm command

19d. Execute "`fmgr`" script file and put the file "`rfc2186.txt`" to the recycle bin. Empty the recycle bin and exit the script. Is the file in the "`.junk`" directory? Why?

No, the file is no longer in the .junk directory. This is because of

```
rm $HOME/.junk/*
```

This command will delete the contents of the .junk directory

19e. Execute "`fmgr`" script file and permanently delete the file "`rfc1034.pdf`". Is the file in the "`.junk`" directory? Why?

No, we permanently deleted the file so it is not in the .junk directory. This is because of

```
if [ -f $mfile ]; then
        rm -i $mfile
```

Notice, we did not even mention the .junk directory. If the file exists, it will be deleted

Although it may look like that shell scripts are like programs, notice that shell scripts are able to execute shell commands too. Most network administrators automate tasks using shell scripts. You can even change server settings, add a batch of users, or even delete a batch of users of using shell scripts.

6.4. Conclusion/Reflection

What other commands you have learned so far might need an alias? (You may include the options for the command)?

I think safety is important when dealing with files, i do not want to deal with losing or accidentally overwriting a valuable file. Here are the alias I would make

cp = cp -i (So that everytime i copy, i have a confirmation message)
rm = rm -i (So that before I delete, I have a confirmation message)
mv = mv -i (So that before I move, I have a confirmation message)